

Universidade do Minho

Licenciatura em Engenharia Informática

LI3 – Relatório do Projeto 1º Fase

Grupo 21

Trabalho realizado por:
Pedro Miranda – A104180
José Lima - A106888
Rodrigo Silva - A108961

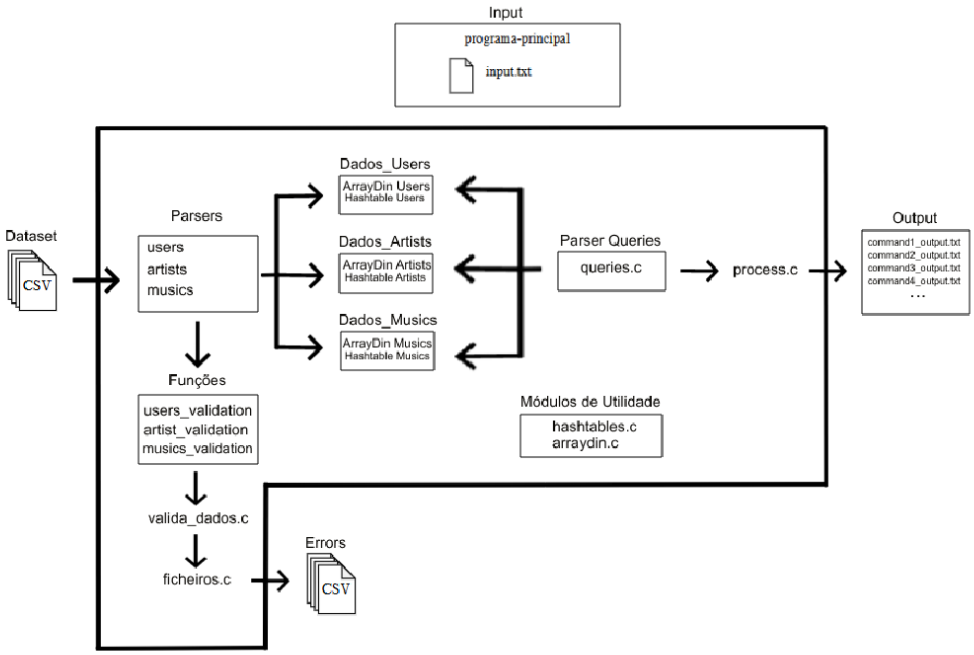
Introdução

Este trabalho prático teve como objetivo principal desenvolver competências na implementação de programas estruturados na linguagem C, com foco na criação de um sistema de streaming de música. O projeto consistiu em explorar um conjunto de dados relacionados a músicas, artistas e utilizadores e os armazenar em estruturas de dados adequadas e realizando consultas específicas sobre essas informações.

Durante o desenvolvimento, demos atenção especial à organização modular do código, separando as interfaces (.h) das implementações (.c), para facilitar a organização e garantir o isolamento das diferentes funcionalidades do programa. A estrutura do sistema e as técnicas utilizadas permitiram uma manipulação eficiente dos dados, incluindo validações específicas e armazenamento otimizado, destacando a versatilidade do programa para processar e responder às queries com precisão tendo sempre em conta os leaks de memória tentando sempre vazar o mínimo possível.

Ás características do programa que julgamos serem particularmente interessantes é o facto que logo após o parsing calculamos a idade, a discográfica total e vemos os géneros que os users mais gostam, invés de depois fazermos uma função que faça isso o que nos faria percorrer mais vezes os users, músicas e artists.

Sistema - Arquitetura



Sistema – Código

programa-principal.c

O programa programa-principal.c é responsável por inicializar e executar o sistema de processamento de dados para um serviço de streaming de música. Ele efetua as seguintes operações principais:

Checagem dos Argumentos de Entrada: Verifica se foram passados os argumentos corretos no terminal para identificar o diretório dos dados (dataset_dir) e o arquivo de queries (caminho_queries). Caso não estejam presentes ou não sejam encontrados o mesmo exibe uma mensagem de erro e por consequente encerra o programa.

Carregamento dos Dados: É responsável por carregar os dados de artistas, músicas e usuários a partir dos arquivos no diretório especificado no terminal. Para cada tipo de dado:

Inicializa as estruturas correspondentes com as funções iniciaArtists, iniciaMusics e iniciaUsers.

Verifica se o carregamento foi bem-sucedido. Se existirem erros (como falhas no carregamento ou dados inválidos), uma mensagem de erro será mostrada, e o programa será encerrado.

Informa ao usuário se foram detetados dados inválidos e caso os detete gera arquivos com detalhes dos erros, como artists_errors.csv, musics_errors.csv e users_errors.csv.

Processamento das Queries: Após carregar os dados, o programa chama a função lerQueries para processar o arquivo de queries fornecido. Essa função executa as consultas solicitadas usando as informações dadas, gerando automaticamente as respostas conforme as instruções do arquivo de entrada fornecido.

Libertação de Memória: Ao concluir o processamento das queries, o programa libera toda a memória alocada para as estruturas de dados de usuários, artistas e músicas, garantindo que ocorram o mínimo de vazamentos de memória possiveis.

Conclusão: Exibe uma mensagem de conclusão, "Programa executado com sucesso", indicando que todos os processos foram executados e finalizados corretamente.

Essencialmente, o programa-principal.c coordena o fluxo de execução do sistema de streaming, carregando os dados, processando as queries e finalizando com a liberação de memória.

users.c

Este ficheiro é relativo à criação das estruturas de armazenamento e processamento destes dados, ou seja, dos utilizadores. Ele contém funcionalidades para acessar e definir informações sobre os usuários, como ID, email, idade, país e tipo de assinatura, através de uma estrutura HashTable eficiente. O código também realiza a validação dos dados do usuário, garantindo que informações como email e IDs de músicas curtidas estejam em formatos corretos.

Além disso, ele permite criar e inicializar dados de usuários a partir de arquivos CSV, tratando erros e organizando os dados em memória. Ao final, há funções para liberar a memória alocada para os dados dos usuários e para exibir informações detalhadas dos usuários para fins de teste.

artists.c

Este código gerencia os dados de artistas, armazenando as informações em uma HashTable. Permitindo procurar e atualizar dados como nome, país, receita por stream e tipo de artista (individual ou grupo). Além disso, o código valida os dados ao carregar de um arquivo CSV e inclui funções para libertar a memória utilizada e para imprimir os dados, auxiliando em testes e garantindo uma gestão de memória eficiente.

musics.c

Este código gerência os dados de músicas no sistema de utilizando uma HashTable para um armazenamento mais eficiente. O mesmo fornece funções para acessar informações da música, como título, artistas, duração, gênero, ano e letra, e para definir ou atualizar esses dados.

Também valida dados de entrada ao carregar músicas de um arquivo CSV, verificando formatos corretos para duração e IDs dos artistas. Inclui ainda funções para libertar a memória alocada e imprimir informações das músicas para teste, assegurando uma manipulação estruturada e segura dos dados no sistema.

valida_dados.c

Este código realiza validações de dados essenciais para o sistema, incluindo verificação de datas, horários e tipos de assinatura. A função data_validation verifica se a data está em formato válido (YYYY/MM/DD) e dentro de um intervalo aceitável. A função hora_validation confirma se o horário está no formato correto (hh:mm:ss) e dentro dos limites possíveis para horas, minutos e segundos.

Além disso, o código inclui a função calculaIdade que calcula a idade com base na data de nascimento fornecida, e converteParaFormatoHora, que transforma uma duração em segundos no formato de tempo (hh:mm:ss). A função verifica_subscription_type valida o tipo de assinatura, aceitando apenas os valores “normal” e “premium”, retornando uma mensagem de erro caso o valor seja inválido.

arraydin.c

Este código define uma estrutura de Array Dinâmico (ArrayDin) e oferece funções para gerir listas de elementos de maneira eficiente. A função inicia_ArrayDin é responsável por alocar e inicializar o array com um tamanho inicial, permitindo a inserção de novos elementos. À medida que o array se expande, a função inserir_ArrayDin ajusta automaticamente o tamanho, dobrando-o quando necessário para acomodar novos itens.

As funções getADN e getADItem permitem, respetivamente, obter o número total de elementos e aceder a um item específico através do índice. Para uma gestão adequada de memória, apaga_ArrayDin percorre e libera cada elemento no array, aplicando uma função de liberação (como free). Especificamente para strings, apaga_ADString e apaga_ADStrings garantem que a memória ocupada pelas strings seja devidamente libertada, evitando possíveis problemas de memória.

ficheiros.c

Este código é uma ajuda valente para mexer com ficheiros em C, ideal para operações como ler, escrever e comparar o conteúdo deles.

A função iniciaFicheiro abre o ficheiro no modo que precisas, e apagaFicheiro encarrega-se de fechar e libertar a memória usada. LinhasFicheiro conta o número de linhas no ficheiro, enquanto escreveLinha escreve uma linha nova. Para leitura linha a linha, tens a lerLinha, que devolve cada linha como uma cópia nova.

Se estás a usar arrays dinâmicos, escreveFicheiro passa o conteúdo de um ArrayDin para o ficheiro, e o parser carrega cada linha do ficheiro num ArrayDin, onde cada linha fica como um item separado. A função separa_String divide uma string em pedaços com base em aspas, e mete cada parte no ArrayDin.

A função compararResultados compara dois ficheiros linha a linha para confirmar se batem certo, mostrando-te os acertos e os erros numa query específica. Já parse_csv_line trata uma linha CSV e divide-a em campos, até mesmo se tiver aspas, para te facilitar o trabalho com dados estruturados.

Assim o código fica mais organizado e ajuda bastante a lidar com ficheiros.

hashtable.c

Este código implementa uma Hash Table que permite armazenar, aceder e gerir dados de maneira eficiente. A função hashFunc calcula o índice do item na tabela com base na string fornecida. A Hash Table é iniciada com a função inicia_HashTable, que aloca e inicializa o número de slots, calculando um valor primo para maximizar a eficiência.

Funções como insere_HashTable e procura_HashTable permitem inserir elementos e procurar um item específico, respeitando colisões através de encadeamento. getHTN e getHTMax retornam o número de elementos e a capacidade máxima da tabela, respetivamente. O método piorCasoHashTable calcula o maior número de colisões num slot.

process.c

Este código implementa a função lerQueries, que lê e executa comandos de um ficheiro de queries. O ficheiro é aberto e, se encontrado, o programa processa cada linha, contabilizando o número de queries lidas, executadas com sucesso, falhadas e desconhecidas.

Cada linha é interpretada como uma query, e a função interpreterQueries é chamada para analisar e executar a query, armazenando o resultado num Array Dinâmico (linha). Se a execução for bem-sucedida, os dados são gravados num ficheiro de saída específico para cada comando.

Tempos de execução podem ser apresentados, e o programa fornece um resumo do processamento após completar todas as queries, detalhando o total de queries lidas, bem-sucedidas, falhadas e desconhecidas.

Query1

A função query1 verifica a existência de um usuário com um ID específico numa HashTable. Se o usuário existir, a função recupera dados como email, nome, apelido, idade e país, formata-os numa string e insere essa string no array dinâmico output. Caso o ID não seja encontrado, insere uma linha vazia no output. A função faz uso de funções de busca e gestão de memória para assegurar a liberação dos dados após a inserção.

Query2

A função query2 procura artistas na estrutura Artist_Dados, filtrando-os por país (caso especificado) e ordenando-os pela duração total da discografia em segundos. Primeiro, armazena os artistas correspondentes num array temporário, ordena-o e converte a duração da discografia para o formato hh:mm:ss. Em seguida, cria uma string formatada com o nome, tipo, duração da discografia e país de cada artista e armazena-a num ArrayDin. Se não houver artistas encontrados, insere uma linha vazia no ArrayDin.

Query3

A função query3 identifica os gêneros musicais mais populares entre os usuários numa faixa etária específica (definida por min_age e max_age). Primeiro, percorre todos os usuários na tabela dados_users, filtrando-os pela idade e acumulando os gostos por gênero numa HashTable. Em seguida, converte a HashTable de gêneros para um array e ordena-o pela popularidade dos gêneros e, em caso de empate, por ordem alfabética. Os resultados formatados (gênero e contador) são então inseridos num ArrayDin. Caso nenhum gênero corresponda aos critérios, insere uma linha vazia no ArrayDin.

programas-testes.c

O programa-testes é uma aplicação que processa dados de usuários, músicas e artistas em um sistema de streaming, utilizando arquivos de dataset e queries como entrada. Ele inicia estruturas de dados específicas para usuários, músicas e artistas, e valida o carregamento dos dados.

Em seguida, o programa executa um conjunto de queries conforme especificado no arquivo de queries, medindo o tempo de execução de cada uma. Para cada query, ele gera um arquivo de saída contendo os resultados, além de comparar esses resultados com os arquivos esperados.

O programa também coleta e exibe informações sobre o uso de memória e o número de linhas corretas e incorretas nos resultados das queries, finalizando com a liberação de memória associada aos dados carregados e exibindo uma mensagem de conclusão.

Conclusão do porque das escolhas dos sistemas

Esta organização do código, com uma estrutura modular e funções específicas para cada componente, é uma abordagem vantajosa, especialmente num projeto de grande escala como um sistema de streaming de música. Aqui estão os principais motivos que tornam esta estrutura eficaz:

Separação de Responsabilidades

- A divisão do código em módulos como users.c, artists.c, musics.c, queries.c, e hashtable.c permite que cada componente do sistema tenha funções e responsabilidades bem definidas, o que facilita a compreensão do que cada parte do sistema faz e minimiza o risco de sobreposição de funcionalidades num único ficheiro.
- Por exemplo, queries.c é responsável exclusivamente pela interpretação e execução de queries, enquanto hashtable.c gere a estrutura de dados de dispersão.

Facilidade de Manutenção e Debugging

- Com o código organizado em módulos específicos, qualquer problema ou melhoria pode ser abordado diretamente no módulo relevante sem impacto em todo o sistema. Se

surgirem erros em users.c, estes estarão associados ao carregamento e manipulação de dados de utilizadores.

- O uso de valida_dados.c para validação centraliza as verificações de formato e consistência dos dados, evitando duplicação de código e simplificando a manutenção.

Reutilização de Código e Expansibilidade

- Funções genéricas e reutilizáveis, como as de gestão de HashTable em hashtable.c, tornam o código mais modular e escalável. Outros módulos que precisem de armazenar e procurar dados podem usar essa estrutura sem reescrever funções de dispersão.
- Este princípio também se aplica ao uso de arraydin.c para listas dinâmicas, facilitando a gestão de listas de dados de tamanho variável.

Desempenho e Eficiência

- Funções como hashFunc e a implementação da HashTable foram desenhadas para lidar de forma eficiente com grandes volumes de dados, o que é essencial para um sistema que processa dados de músicas, utilizadores e artistas.
- A medição do tempo e da memória usada em programa-testes.c permite monitorizar e ajustar o desempenho do sistema quando necessário.

Facilidade de Testes e Verificação de Resultados

- Com a organização do código através de programa-testes.c, é possível executar testes específicos e comparar os resultados das queries, automatizando a verificação da precisão das operações. Isto torna o sistema ideal para desenvolvimento iterativo e melhorias contínuas de precisão e eficiência.

Em suma, esta estrutura modular promove um desenvolvimento mais organizado, eficiente e fácil de expandir. Assim, o código mantém-se sustentável e adaptável a alterações e melhorias futuras, aspetos essenciais em projetos de software complexos.

Discussão

Análise do Trabalho

Ficamos satisfeitos com o resultado final do trabalho conseguimos manter-nos organizados e trabalhamos em conjunto para chegar ao melhor resultado possível, tivemos dificuldade em diminuir o vazamento de memória e acabou por ser o único parâmetro que não conseguimos diminuir o mínimo possível apesar dos esforços mas no geral estamos contentes com o nosso desempenho e satisfeitos com os resultados.

Análise do Desempenho

```
Média de tempo para a querie 1: 0.000019 segundos
Média de tempo para a querie 2: 0.000069 segundos
Média de tempo para a querie 3: 0.129701 segundos
Tempo total de execução: 3.317117 segundos
Uso de memória: 716764 KB
Linhas corretas: 320
Linhas incorretas: 0
Programa-testes executado com sucesso.
```

Ambiente de Testes: Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz 2.59 GHz

```
Média de tempo para a querie 1: 0.000024 segundos
Média de tempo para a querie 2: 0.000180 segundos
Média de tempo para a querie 3: 0.309135 segundos
Tempo total de execução: 7.746865 segundos
Uso de memória: 716544 KB
Linhas corretas: 320
Linhas incorretas: 0
Programa-testes executado com sucesso.
```

Ambiente de Testes: Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz

```
Média de tempo para a querie 1: 0.000016 segundos
Média de tempo para a querie 2: 0.000029 segundos
Média de tempo para a querie 3: 0.081915 segundos
Tempo total de execução: 2.053320 segundos
Uso de memória: 716804 KB
Linhas corretas: 320
Linhas incorretas: 0
Programa-testes executado com sucesso.
```

Ambiente de Testes: 12th Gen Intel(R) Core(TM) i5-12400F 2.50 GHz

Conclusão

Na conclusão deste trabalho, foi possível consolidar várias competências fundamentais no desenvolvimento de programas em C, como a gestão eficiente de estruturas de dados e a importância da modularidade e da organização do código em grandes projetos. A implementação de módulos específicos para diferentes componentes do sistema (utilizadores, músicas, artistas) permitiu uma estrutura mais clara e facilitou a manutenção do projeto. Este trabalho reforçou o valor da separação de responsabilidades em grupos, algo essencial para manter a clareza e a organização